Hope Beacon: The Complete Handbook

Everything needed to run Hope Beacon, move it to a new project, and keep it working. Written for the people who run a church, and for the AI tools that will be asked to continue the work.

Version: 1 September 2026 (evening) · **Applies to:** migrations through `20260901140000` · **Licence:** AGPL-3.0 · **Source:** `github.com/Klydo131/Open-Sentry-Beacon`

> **NOTE** · How to read this
>
> **Running a church?** Parts 1 to 5 are yours. They assume no technical knowledge and nothing needs installing.
>
> **Setting the app up, or moving it?** Parts 6 to 12. Follow them in order; each one checks the one before it.
>
> **An AI tool picking this up?** Part 13 is written for you and states the invariants you must not break.
>
> **Handing this to a member rather than an administrator?** Give them `HOW-TO-USE` instead. It is the same app explained in pictures, phone-shaped, with nothing in it about databases or deployment.

1. What the app is

Hope Beacon is a discipleship app for one church. A member of that church walks alongside one other person at a time, and the app carries what that takes: the conversation, the readings, the prayer requests, and a quiet record of how far along the journey somebody has come.

Three things define it, and every decision in the rest of this handbook comes back to one of them.

It is invitation only

There is no public sign-up and there never will be. Somebody at the church enters a name and an email address, the app sends an invitation, and that is the only door. A stranger who finds the web address sees a sign-in screen and a tutorial, and nothing else.

A conversation belongs to two people

What an Explorer says to their Guide is readable by those two and nobody else. Not other Guides, not Directors, not the person who owns the server. The only exception is a safeguarding report, which a Director may read in place, and the app says so plainly on the screen where a report is made.

A message can be corrected, or taken back

You can edit or delete your own messages, and only your own. An edited one is marked **edited**, because the other person read the first version and a silent rewrite is a way to make somebody doubt what they remember. A deleted one leaves a line saying **"You deleted a message"** or **"Maria deleted a message"** — never a gap, because a message that vanishes without trace reads as one that was never sent, and the other person is left believing they imagined it.

Deleting removes the words from the conversation, not from the record. What the message said is kept where neither of you can read it, and a Director sees it only when a safeguarding report is being looked at. This is the same rule as the Guild Room, where a reported post's words are copied into the report before anybody can remove them: the first thing somebody does after being reported is delete. Take back a message you regret and it is gone from the screen; it is not gone from the account of what happened.

> **CAUTION** · Until 9 September 2026, either person could rewrite the other's words
>
> The rule that lets a recipient mark a message as read was written in a way that also permitted editing the whole message, including one you did not write, with nothing shown on screen. Nobody appears to have used it. It is closed, and the conversation is now the one thing in this app that only its author can change.

Nobody can flood a room

One account can send forty messages a minute and no more, across the conversation, the Guild wall and the Guides' room. That is two a second held for a full minute, which nobody types; it is set to stop a script rather than a person, and somebody who somehow reaches it is told plainly to wait a moment rather than shown an error code.

It bounds the machine, not the behaviour. Forty unkind messages a minute is still forty unkind messages, and the answer to that is the report route and a Director — the same answer as everywhere else in this app. No counter substitutes for a person whose job it is to look.

The limit is people, not computers

A Guide walks with at most five Explorers at once, and the database enforces it rather than the screen. The app will run a church of a hundred without complaint, on the plan it is on today and with room to spare. See What it costs (https://github.com/klydo131/open-sentry-beacon/blob/main/docs/./WHAT-IT-COSTS.md) for the measured figures. What it cannot do is find you

a sixth Guide. Growth here means recruiting and training people, and the app is built to keep that constraint visible rather than hide it behind a number that keeps rising.

2. The four roles

A person's role is chosen when they are approved, and it decides everything they can see for as long as they are in the church. **Nobody can change their own role, including the Executive Director.** That is enforced in the database, not in the app.

Role	What they do	What they can see
Explorer	Walks the journey. Reads what their Guide sends, talks with them, asks for prayer.	Their own journey, and their conversation with their Guide. Nothing about anybody else.
Guide	Walks with up to five Explorers. Chooses what to share and when. Recommends new people, but cannot invite them.	Only the Explorers paired with them. Never another Guide's people.
Director	Runs the church. Invites, approves, pairs, and reads safeguarding reports.	Everyone in their church and the counts behind them. Not private conversations, except inside a report.
Executive Director	Oversees one or more churches, and appoints Directors.	Everything a Director sees, across every church they oversee.

> **NOTE** · The Head Executive Director
>
> One account is the root of authority. It cannot be suspended or removed by anybody, including itself, so a church can never lock itself out of its own app. Guard the password for that account the way you would guard the keys to the building.

What an Explorer's shelf opens with

The starter shelf is the same twenty resources for everybody, but an Explorer is shown eight of them first, in this order:

> A Bible they can read on a phone · Jesus 101 · The Desire of Ages · Steps to Christ · BibleProject · Discover Bible Guides · What Seventh-day Adventists Believe · Sabbath School this quarter

One doctrinal page, deliberately. Somebody deciding whether to follow Jesus is not helped by being handed the twenty-eight fundamental beliefs and the full prophetic history on their first day, and the Guide walking with them is the right way to meet the rest.

Nothing is hidden. The Great Controversy, the collected writings of Ellen G. White, the quarterly archive and the General Conference publications are all still in the shelf for everyone else, and for an Explorer the moment they go looking. What changed is only what is put in front of somebody first.

The shelf is a constant in `lib/starter-kit.ts`, not a table, so it costs no storage and no query. `tests/the-explorer-starts-with-jesus.mjs` keeps the short list short. The twenty links point at other people's websites, so `links.yml` checks weekly that every one of them still opens.

3. The journey

Five stages, and an Explorer moves through them at their own pace. The stage is a note for the Guide, not a score, and nothing in the app hurries anybody along.

Stage	What it means
Beginner	Just beginning. Somebody has said yes to being walked with.
Connect	Building rapport. Getting to know each other.
Care	Walking alongside. The longest stage, and usually the most valuable.
Call	A point of decision.
Cultivate	Growing in faith after that decision.

The first stage was called *Create* until August 2026. That named what the church was doing; *Beginner* names where the Explorer is, which is whose journey it is. Nothing changed in the database, so no history moved.

A sixth idea sits behind these: **Commission**. An Explorer who has been walked with becomes a Guide, and walks with somebody else. That is the whole point of the design, and it is the only kind of growth that does not run out.

4. Running it, week to week

Inviting somebody

Step 1. Sign in as a Director, open **Admin**, then the **Approvals** room.

Step 2. Enter their name and email address, choose the role, and press **Send invitation**.

Step 3. They receive an email written for that role. An Explorer, a Guide and a Director each get a different message, because each is being asked for something different.

Step 4. They choose a password. That is what finishes the sign-up.

Step 5. They appear under **Awaiting approval**. Approve them, and they can enter.

> **CAUTION** · One live invitation per person
>
> Sending a second invitation to the same address switches off the first. If somebody says the link does not work, ask whether they have two emails, and tell them to use the newest one. Never open somebody else's invitation link yourself: it works once, and opening it signs you out and starts their sign-up on your device.

Pairing a Guide with an Explorer

Open the **Pairings** room, choose one of each, and press **Create pairing**. They can talk from that moment. A Guide already carrying five will not appear in the list, because the database will not allow a sixth.

Five is a ceiling, and a church can raise it. A Guide walks with five Explorers by default, enforced by the database. A congregation with more Explorers than Guides can carry may raise that limit in Church settings, up to twenty-five. Only a Director or an Executive Director can, and only for their own church: a Guide can never give themselves more people.

Raising it is a decision, not a drift. Five is the number the design is built around because it is how many people one person can actually walk with.

The one number worth watching is unpaired Explorers. An Explorer with no Guide has been invited into an app where nothing happens. Your dashboard opens on that number for exactly this reason.

Your Guide is a real person

An Explorer's screen opens with the person walking with them: their picture, their name, their city and what they said they care about. All of it is what that Guide typed on their own profile.

It was a name on a line, and everything else on that screen is generated (the greeting, the stages, the notices), so a name in the same typeface as the rest proved nothing. Somebody who has been handed a stranger's name by an app has no way to tell whether anybody is really on the other end, and the whole product rests on them believing there is.

> **IMPORTANT** · Ask your Guides to set a picture
>
> Almost none have one. Where there is no photograph and no icon the card falls back to two initials on a coloured circle, which is exactly the problem it was built to solve. A Guide's own screen now asks them for one and stops asking the moment either is set. An icon is a single tap, on **Profile**.

Nothing else about the Guide reaches the Explorer: no birthday, no contact details, nothing anybody else recorded about them.

Finding one person

The approved list gets long. Once it passes five accounts a search box appears above it: type any part of a name and the list narrows as you type, showing "4 of 37" so a short list is never mistaken for a lost account.

Past six accounts the list gets a scroll bar of its own rather than growing down the page. Forty accounts is roughly four thousand pixels, and every control above the list, the search box and the bulk buttons, used to scroll out of sight the moment somebody started reading names. Now the roll and the things you do to it stay on screen together.

The "New" mark

Anybody who finished signing up in the last seven days carries a green **New** badge on the rosters and on a Guide's cards. It works out the answer from the date every time it draws, so it can never be left on somebody by mistake, and it counts from when they chose a password rather than when a Director typed their address.

The badge gives a soft ring twice when the screen draws, then stops. A mark that pulses forever is a mark people stop seeing, and then it is only noise on somebody's name. Every role gets the same badge, which is deliberate: a different mark for a new Explorer would tell everybody who can see the list which members are Explorers, and the app takes care elsewhere not to say that.

A device set to reduce motion gets a still outline instead of the ring.

Disapprove, or delete

These are different acts and the difference matters.

	Disapprove	Delete
What happens	The account is switched off. They cannot enter, but they and their history stay.	The account, its messages and its pairings are removed for good.
Reversible	Yes. Approve them again.	No.
Their email	Still in use by that account.	Freed. They can be invited again as a brand new member, in any role.
Use it when	Somebody is away, or you are looking into something.	Somebody has left, or an account was created by mistake.

Delete asks a second time in the row itself rather than through a browser pop-up, because on a phone that dialog appears under the thumb that just tapped Delete and the button dismissing it is the one that agrees. It says what will go before it goes. The removal is recorded in a log that outlives the person it describes, so a church can always answer who removed whom and when.

The dangerous buttons are smaller and red, and they are the only red in the app. Delete, Remove, Disconnect and Disapprove are drawn one size down from an ordinary button, in red on white rather than filled. They are still a full touch target, because discouraged is not the same as fiddly, but they no longer look like the thing to press.

A Director asked for this. Disconnect was larger than the thing beside it and read as the more inviting of the two. The thing beside it was not a button at all: *Connect* is the name of the second stage of the journey. The screen now says **Stage · Connect** in that stage's own colour, and a status can no longer be mistaken for an action.

Both presses of a two-step removal are red: the first says the path is dangerous, the second is the act itself. That was backwards in one place, where the harmless first press was red and the irreversible confirmation was grey.

Acting on several accounts at once

Every row in the approved list has a tick box. Tick a few, or use **Select all**, and two buttons appear: **Disapprove selected** and **Delete selected**.

Three things about it are worth knowing before you use it on twenty people.

- **Select all means what you can see.** With a search showing four of thirty-seven, it takes those four. It never quietly reaches the rows the search is hiding.

- **The confirmation names everybody.** Not "delete 12 accounts", but the twelve names, because there is no way to check afterwards and no way to undo.
- **One refusal does not undo the rest.** If the database refuses one person, for instance a Director you may not act on, everybody else is still done and you are told which one failed and why.

> **NOTE** · Who may delete whom
>
> Decided by the database, not the screen. An Executive Director may act on anyone in a church they oversee. A Director may act on Guides and Explorers only, never another Director. Nobody may act on themselves, and nobody at all may act on the Head Executive Director. If an action is refused you are told why, in a sentence.

Meetings

A Guide and an Explorer arrange a time together on the same card, and both see it. Either may propose one: a title, a date and time, and online or in person. The other person confirms it, and either can cancel. Nobody else in the church sees any of it.

The card sits with the conversation, on both sides. On the Explorer's screen it always has. On the Guide's it used to sit under **Journey**, which is the one tab the Explorer never sees — it holds the six stages and the Advance button, and it is where a Guide records where somebody has got to. That put a thing the two of them do *together* inside the folder of things the Guide does *about* them: an Explorer proposed a time from their chat, and their Guide had to leave the conversation and know which tab to open before they could see it had been asked. It is beneath the thread on both screens now, in the same place on each.

In person asks where, and will not let you skip it. The place becomes an **Open in Maps** button that opens whichever map app that person already has, signed in, with their own saved places. Give it a place and a street, for example *Church cafe, 12 Rizal St, Cavite*, because "Church hall" on its own maps to every church hall in the country.

> **NOTE** · A link, never an embedded map
>
> An embedded map needs a Google Maps key, which means a billing account and a key that reaches every browser in the congregation. A link costs nothing, needs no account of ours, and opens the app people already use.

Online asks for the link, and turns it into a button. Paste a Zoom, Meet, Teams, Whereby, Jitsi, Messenger or Skype address and the card shows **Join on Zoom**, named after whichever service it recognises, so the person tapping it knows what is about to open. An address it does not recognise still works, and reads **Join the meeting**.

Before this, an online meeting had a title and a time and nowhere to put the address, so the link went into the conversation as a message and slid up out of sight, and the older the meeting, the further up it had gone.

> **CAUTION** · Only http and https become a button
>
> A meeting link is text one member types and another taps, which is the exact shape of an attack. Anything that is not an ordinary web address is shown as plain text and is not tappable, whatever it claims to be.

The library, and who watches it

Anybody in the church can put a link in the library and share it. A Guide and an Explorer both can, without asking a Director first. That freedom is the point: somebody who finds a reading worth passing on should be able to pass it on.

The library holds links, and files stay on your own device. A file saved under *On this device* is passed from your phone to theirs through your phone's own share sheet, so it never sits on a server. That is what keeps this app free to run while it is small, and storing files for everybody is on the list for when it can be paid for properly. The screen says so, so nobody hunts for an upload button that is not there.

What a Director sees, and what an Executive Director sees

Freedom to share is not freedom from oversight, and the oversight is a record afterwards rather than a gate in front of every share.

You are	You see the record for	You do not see
A Director	Guides and Explorers in your church	Other Directors, or an Executive Director
An Executive Director	Directors	Guides and Explorers

Each rank watches the rank below it and no further down, which is the same shape as the security audit. The record is in **Admin → Security**, under the audit, and shows who added or shared what, with whom, and when. Nothing from a conversation appears in it.

> **IMPORTANT** · The record is kept for 30 days and then deleted
>
> It is there to answer "what happened recently", not to be an archive, and a file of everybody's reading kept forever is a different and worse thing. **If something in it needs to outlast the month, raise a safeguarding report about it.** Those are never deleted, and that is the difference between the two.

Stopping somebody

A Director can **block a Guide or an Explorer** from sharing anything. An Executive Director can block a Director. Nobody can block themselves, and nobody can block upward or sideways; the database refuses it rather than the screen hiding a button.

Blocking takes away the library and nothing else. The person keeps their account and their conversation, and can be let back in with one press. It is the right answer for somebody misusing the shelf, and the wrong answer for somebody who needs a safeguarding report or a case.

Prayer

An Explorer asks for prayer on their own screen, at the foot of it. It goes to the Guide walking with them and to nobody else. There is no audience to choose, and the choice to broadcast one was taken out on purpose.

The Guide presses "I'm praying" and the Explorer is told. Not that the request was read, and not the words of it: the Explorer sees *"Your Guide is praying for this"* with the date, on their own copy of the request.

The date is not decoration. "Somebody is praying about my mother" is worth knowing the day of, and an Explorer who wrote something hard and saw nothing change had no way to tell whether anybody had seen it at all.

> **NOTE** · What the notice does not carry
>
> The Explorer's own words are never repeated back to them in the notice, and nothing about the request leaves the two of them. If the person pressing the button is the person who asked, nothing is sent at all: nobody is notified about themselves.

A Guide's own screen puts the requests waiting for them at the top, before the roster, with a mark on each Explorer who has one open. The mark clears when they press it, which is what keeps it worth reading instead of becoming permanent furniture.

Lesson studies

A Guide writes their own. Create a series, add studies to it, attach the handouts you already use, and publish it when it is ready. Until you publish, only you can see it. Anybody in the church can then open the series, read the studies and open the files.

Directors keep the same control over everything, which is what running the church means. A Guide may edit and delete only what they wrote.

This works on a phone, and for a while it did not. The writing desk is in the Office, the Office was only ever linked from the left column, and the left column does not exist below the width of a laptop. So a Guide on a phone or an iPad held upright could read studies but never write one, with nothing on screen to suggest the room existed. Three rooms were in that state (Office, Publish and Cases) and the fix was to put them in the header row that a phone actually has. There is a check now that fails the build if a room is added to one list and not the other.

Publish

Everything you write for other people to read is in one room, and every role has it. Writing used to be scattered across the screens people *read*: the blog desk sat on an Explorer's journey, on a Guide's Office and inside a Director's admin tab, and the announcement composer sat on top of the church home screen, which is the page somebody opens to find out what the church has said.

Publishing is a task, and a task gets a room.

An **Explorer** writes a blog post here, which the whole church reads. They cannot pin an announcement, and the screen says so and points them at the blog rather than showing them a blank space. A room that is empty for a whole role reads as broken.

Taking a notice down still happens on the church home screen, beside the notice itself. Deleting is about the thing in front of you; writing is something you go and do.

Announcements

A notice pinned where the church will see it: an icon, a title, a line of detail, and a free-text when, because "This Sabbath, 9:00 AM" and "Every evening this week" are what a church actually writes and neither is a date. Notices come down by being taken down, not by a clock nobody set.

Guides, Directors and Executive Directors can write one. Guides were left out at first, which meant a Guide arranging something for the five people they walk with had nowhere to pin it and sent the same message five times.

Explorers cannot. A notice sits above everybody's church screen, and pinning something there is an act of leading rather than of speaking. An Explorer with something to say to the church has Community Blogs, which is exactly that and does not sit above everyone else's.

Every notice goes to the whole church, and there is no audience to choose. A private option existed briefly and was taken out: a notice only part of the congregation can see is not a notice, and having the setting invites somebody to use it. The screen says so above the Post button, so nobody writes one assuming it will reach fewer people. Anything meant for fewer people belongs in a message or in Community Blogs.

Anybody in leadership can take down any notice, and an author can take down their own. A pinned notice reaches the whole church, and anything that reaches the whole church needs an off switch that does not depend on the person who wrote it being available.

Where a notice appears depends on whose screen it is, because the two jobs are different:

- **Guides, Directors and Executive Directors** see notices first, under the greeting. Their job is the church, and a Guide in particular carries the notices onward to the people they walk with.
- **An Explorer** sees them after their Guide's card and before the conversation. An Explorer opening their journey is looking for their person, not for the church; putting the church's notices above that answered a question they had not asked. Put below the conversation, they would never have been scrolled to at all.

Nothing is drawn at all when nothing is pinned, so an ordinary day costs no space on any screen.

Community Blogs

Anybody approved in the church can write a post. Explorers, Guides, Directors and Executive Directors all have **Your blog** on their own screen. Before tonight only Guides and leaders could, and an Explorer who tried was shown an error from the database.

Three audiences, and the choice is made before publishing.

Audience	Who reads it
Everyone in the church	Every approved member of your church. It appears in Community Blogs.
Only the people I walk with	For a Guide, their Explorers. For an Explorer, their Guide.
Only the people I choose	The people named on it, and nobody else. It only appears when there is somebody to choose.

A post stays private until it is published, and **Make private** takes it back off without losing it.

> **CAUTION** · A church-wide post is signed
>
> Choosing **Everyone in the church** puts your name and your role on it, and the screen says so before you press publish. That is on purpose: a blog everyone reads where some posts are signed and others are anonymous is one nobody can hold to account. The narrower audiences follow the app's ordinary rule, which does not name an Explorer's role to people who have no reason to know it.

Community Blogs sits below the masthead on the church Home screen, and at the bottom of every other role's own screen, newest first. It is deliberately never the first thing anybody sees. On the church Home the order is the church's name, then the blogs, then the pinned notices; on a Guide's or an Explorer's own screen the blogs are last, because what a person came there to do belongs above what everybody else has written.

It draws nothing at all when nobody has published, rather than leaving an empty card on the screen.

Past three posts it gets a scroll bar of its own instead of growing down the page, and **Hide** folds it away entirely. Both are remembered on that device, so somebody who would rather not read the blogs shuts them once rather than scrolling past them every time. Folded, the heading still says how many posts are waiting, because a shut panel with no count looks like an empty one and nobody opens it again.

On an Explorer's **My Journey**, the first thing on the screen is their Guide's name. The whole design says the journey is a relationship, and an Explorer opening that screen is looking for their person.

Directors and Executive Directors can delete any post in their church. That is not tidying up; it is the reason an audience open to every member is safe to have at all. A church-wide megaphone with no way to switch it off is a problem waiting for a Sabbath morning. Anybody can always delete their own.

The Guild Room

A guild is a named group inside the church, made by a Director: a Bible-study cohort, a campus, a language, a Sabbath afternoon team. A pairing is one Guide and one Explorer, which is the right shape for discipleship and the wrong shape for everything a church does in groups. Only Guides and Explorers are put into one; Directors run guilds rather than belonging to them.

Everybody in the church can see that a guild called *Palawan Campus* exists. **Who is in it is visible to Guides and leadership only**, because handing an Explorer a list of the other Explorers would turn a set of private relationships into a public roster of everybody being discipled here.

The **Guild Room** is that group's shared board: Guides and the Explorers in it, together. Four kinds of thing go on it: an **encouragement**, a **study note**, a **prayer**, or a way the guild can **care** for somebody. Anyone in the guild can say *Amen* to a post.

It shows no names. A post is signed *You*, *A Guide*, or *A fellow Explorer*, and nothing else. The board never publishes who is in the guild, which is what makes the room worth having: a group can talk without it becoming a roster of everybody's Explorers.

The board updates on its own. A post or an *Amen* from anybody in the guild appears on everyone else's screen within a second or two, with no refresh. This was the last room in the app that still needed one, and it was left until last because the obvious way to do it would have broken the paragraph above: making the posts readable by the

browser would have handed over who wrote each one, which is precisely what the room exists not to do. Instead the app watches a separate signal that records only *that this guild's wall changed* — no author, no words, nothing about a person — and then re-asks for the board through the same route as always, labels and all. **Nothing about how little you are told changed; only how quickly you are told it.**

Directors and Executive Directors are not in it. A group talking honestly is what the room is for, and a Director reading over their shoulder is a different product. Guild membership itself is still managed by a Director, from the Church room.

Anybody can report a post, and a Director can take it down. That is the one way leadership sees into the room, and it opens only when somebody reports something:

- **Report this post** sits under every post that is not your own. It does not ask who wrote it, because you do not know and should not be told. The app works that out on its own and never shows you the name.
- The report lands in the same **Safeguarding** queue as everything else, and every Director is notified.
- **What the post said is copied into the report.** If the person who wrote it deletes it afterwards, which is exactly what somebody who has just been reported does, the Director still reads the words.
- A Director can **delete the post**, and that removal is written into the security audit before the post goes, so it cannot be lost.

> **CAUTION** · This room shipped without any of that
>
> For a day the board had no way to report a post, no way for a Director to see in, and nobody but the author could delete anything. Explorers are in these guilds and some Explorers are children. Every other place in Beacon where one person can be hurt by another has the same three things on the same screen: a way to report it, somebody whose job it is to look, and a record that outlives the person it describes. **Apply that test to any new room before it ships, not after.**

Undoing a step

Advance stage is one tap, and taps go wrong. **Undo, step back** sits beside it and puts an Explorer back a level. It asks first, and it is recorded as a correction rather than erased, so the history stays honest. The Explorer is never shown their stage either way, so a correction is invisible to the person it is about.

Numbers a Guide can see

The screen a Guide lands on opens with their own figures: how many Explorers they have, how many have **graduated**, how many are still walking, and the breakdown by level. Above that sits whatever is waiting today, which is usually short: prayer requests, and the next meeting with a name and a day.

Graduated means reached Commission: walked the whole journey and now sent to walk with somebody else. It is the number the whole design exists to produce.

Rooms and subrooms

A room is a folder, and a subroom is a folder inside it. Open a room and a row of subrooms sits across the top; tap one and you are in it. Nothing else is drawn, so there is nothing to scroll past.

Six rooms work this way now. Measured on a phone, with the sample church in them:

Room	Was	Now
The Library	11 screens of scrolling	3 folders: Browse, Featured, On this device
Settings	7 screens	5 folders: Install, Alerts, Language and size, Church, Help
My Journey, an Explorer's own screen	7 screens	4 folders: My Guide, Study, Church, Prayer
The Church	5 screens	3 folders: Notices, Community Blogs, The numbers
My Explorers, a Guide's home	4 screens	4 folders: My Explorers, Follow-ups, Prayer, Church
The Office	3 screens, and nine cards	5 or 6 folders, below

Publish, Cases, the Guild Room, Mail and Profile are left alone. They are one or two screens and mostly one thing; a row of choices above a single card is furniture, not navigation.

The Office

Guides, Directors and Executive Directors have an **Office**, in the left column on every screen. It holds the work: the numbers, the downloads, the studies you write and the shelf you stock. Its subrooms are:

A Guide's subrooms	A Director's subrooms
Lesson studies, Resources, Guides' room, Put a name forward, Numbers	Numbers, Reports, Lesson studies, Library, Pairing requests, Guides' room

Three things about it are worth knowing:

- **It opens where your work is.** A Guide lands on Lesson studies, because writing is what a Guide comes here to do. A Director lands on the numbers.
- **It remembers where you were.** Somebody who lives in Lesson studies lands there tomorrow, and a Director's habit is their own rather than everybody's.
- **A link still beats the habit.** *Guides asking to pair* on the desk opens the Office already on Pairing requests, whichever subroom you were last in. Being sent to the right room and left to find the shelf is the same as not being sent.

The count beside **Pairing requests** is how many Guides are waiting on an answer. Without it a Director has to open the subroom to find out whether there is anything in it, which is the scrolling problem again with a tap on top.

> **NOTE** · Why this room needed it most
>
> It held nine panels down one page. A Guide who came here to write a study passed their numbers, the shelf, two pairing cards and a recommendation form on the way, every single time, and on a phone that is most of a minute of thumb. The Director's screen has worked in rooms since it was split up; the Office simply never got the same treatment, and neither had anywhere else.

> **NOTE** · Two things that did not move
>
> **An Explorer's way out of a conversation is on the same screen as the conversation.** Report sits in the My Guide folder with the thread, the Guide's card and the meetings, because the journey is a relationship and splitting it across folders would be the worst thing this change could do.
>
> **A Guide still sees the church's notices before choosing a folder.** They sit above the row, not inside one, because a Guide carries the notices onward to the people they walk with and was told to see them first.

The split is by kind of work rather than by rank. A roster, a conversation, a case is about a person, and lives on that person's screen. Numbers, exports, writing and stocking a shelf are office work, and live here. Before this, a Guide's roster carried study-writing, library-stocking and a blog desk underneath the list of five people they walk with, and a Director's analytics sat three clicks inside an admin tab. The people screens were four screens long and the tools were hard to find.

Follow-ups and prayer requests stayed on the Guide's roster, because they are about the people on it.

Explorers do not have this room, and not because anything is hidden from them. None of it is theirs to do: no roster to report on, no shelf to stock, nobody to write studies for. A room that would be empty for them tells them they are missing something.

Asking to walk with somebody

A Guide can see the Explorers nobody is walking with yet, and press **I have room** with an optional note. It goes to the Directors.

It puts your name forward; it does not make the pairing. A Director still creates it on the Pairings screen, where the limit of five is checked. A button that quietly created a relationship from a list of requests is how somebody ends up with six people.

The Explorer is never told they were asked for. Being wanted and not chosen is not something anybody should have to read about themselves.

> **NOTE** · This widens what a Guide can see, on purpose
>
> A Guide can normally read exactly two accounts: their own and the Explorer they walk with. You cannot ask to walk with somebody you cannot name, so a Guide now sees the **name** of any Explorer in their church who is waiting. Nothing else comes with it: no birthday, no contact details, no stage, no messages, no prayer requests, no notes.

The Guides' room

A place for Guides and their Directors to talk to each other. Explorers cannot see it. Every conversation in the rest of the app is one Guide with one Explorer, which is right for that relationship and leaves a Guide with a hard week entirely alone.

> **CAUTION** · A room, not private messages
>
> Everybody in the room reads everything in it, and that is what makes it safe to have rather than a limitation. Guide-to-Guide direct messages would be a second private channel with no oversight, in an app whose whole design is that private conversation happens in one place and can be reported. Anyone can delete their own message; Directors can delete any.

Cases

Cases have a room of their own, in the left column on every screen, for every role. A case is a formal proceeding about a person, sometimes about the person reading it, and it used to be a card partway down a dashboard: easy to scroll past on the one day it mattered, and sitting in the same visual rank as a study plan.

The link is always there, whether or not anything is open. A link that comes and goes is one nobody trusts is there, and its absence on a quiet day looks the same as it being broken. When there is nothing, the room says so.

An **Explorer** has the room too, and that matters most. An Explorer called into a case is the person in it with the least standing, and their answer has to be findable without anybody having to tell them where to look. They can write in it even while suspended: suspending somebody pending a hearing must not take away their side of it.

On a Guide's **Care** tab for one Explorer, prayer requests are always shown, even when there are none. The card used to disappear when empty, so the tab held only private notes and read as though a Guide could not see prayer requests at all.

A Director judging a case still works through it in **Admin → Safeguarding**, beside the reports the cases came from. That is a different job from answering one, and the screen tells the two apart by itself.

Safeguarding

Anybody can report a conversation. When they do:

- Every Director of that church is notified at once.
- A Director can read the conversation in place, with what came before and after, rather than as a single quoted line.
- **The person reported is never told.** No message, no notification, nothing they could notice.
- The reporter's name is visible to Directors, because a Director cannot support them or tell a genuine concern from a grudge without it.
- Reports are never deleted, whatever is decided.

A post on a guild board is reported the same way and arrives in the same queue, marked *Guild Room post*, with the post quoted underneath. The Director can close the report as usual and can also **delete the post**. Only leadership of that church can take a post down, and the removal is recorded in the security audit.

The quoted text is a copy taken when the report was made, so it is still there after the post is gone, including when the author deleted it themselves.

The security audit

Admin → Security is a plain list of things that happened to accounts: a name changed, a detail changed, a safeguarding report raised, somebody suspended, restored, removed, approved or disapproved, and a guild post taken down. Each line says who it was about, what happened, when, and how serious it is.

It carries no conversation and no file. Nothing anybody wrote to anybody else appears here, and the screen says so. It is a record of *administration*, not of speech.

Who sees whom follows rank, and only in one direction:

You are	You see activity about
A Director	Guides and Explorers in your church
An Executive Director	The same, plus Directors

A Director is not shown which *leader* acted on something, and those lines read *Church leadership*, because a Director reading a log of another Director's decisions is oversight pointing the wrong way. An Executive Director sees the names.

It lives inside Admin rather than as a room of its own, because it is a leadership tool and a door in the room list that only two roles can open is a door most of the church is invited to rattle.

Reading the numbers

The Church screen opens with four headline figures: Explorers, Guides, Graduated, and how many are waiting for a Guide. Under them sit two charts, each answering one question a Director actually asks.

Who is using it. One panel for Guides and one for Explorers, each showing everybody on the roll for today, this week and this month. The blue part of the bar is the people Beacon recorded doing something; the brown part is the rest.

> **CAUTION** · What "active" does and does not mean
>
> **It is not a count of visits.** Beacon does not record when somebody opens the app, so a number claiming to be visits would be invented. Active means the app recorded them doing something: sending a message, a step on a journey, arranging a meeting, or writing a post or a study.
>
> An Explorer who met their Guide for coffee and wrote nothing down is in the brown part. That is not a failure and it should not be treated as one. What is worth acting on is a whole month of brown for one person, and *Waiting for a Guide* above zero.
>
> Today will almost always look low, and that is the day, not the church. Read the week and the month.

Who is arriving. Four small charts, one for each role: Executive Directors, Directors, Guides and Explorers. Choose the period from Daily, Weekly, Monthly, Quarterly or Yearly. All four share one scale, so a tall bar means the same number of people wherever it appears.

Somebody counts as arriving on the day they finished signing up, not the day their invitation was sent. An invitation that sat unopened for three weeks would otherwise land on the wrong week.

Underneath, in the same period, is what the church decided about people: how many were let in, turned down, suspended, had a suspension lifted, and removed.

> **NOTE** · Removed and deleted are one number
>
> In Hope Beacon, removing somebody from the church deletes their account. There is no separate state where a person has been put out but still has a login. The record of the removal survives them, which is the point of keeping it.

> **NOTE** · Average and middle period
>
> Both are shown because they disagree in the case that matters. One busy week after a quiet month pulls the average up; the middle week is closer to an ordinary one.

Five ways to take the numbers with you, and each says what it is for.

Format	Use it when	Opens in
Sheets	You want to work on the numbers yourself. Keeps both tables and their headings.	Excel, Google Sheets, Numbers
Document	You are sending it to somebody who will edit it.	Word, Google Docs
PDF file	You are sending it to somebody who should not edit it. Downloads straight away.	Anything
Print	You are handing round paper at a meeting. The dialog can also save a PDF, and lets you pick the paper.	Printer, or save as PDF
CSV	You are feeding it into another program.	Anything at all

Every file carries the same explanations, not just the figures: what "active" means and does not mean, that "nothing recorded" is not the same as idle, and that removed and deleted are one number. A spreadsheet with a column headed *Active* and no definition beside it is how somebody decides that eleven of nineteen Guides are not working.

Every file also carries your church's name and the date it was made, and nobody's name is in any of them.

Every one of these numbers is a count. No name, no message and no prayer appears on this screen or in the file, and the part that counts messages runs inside the database and hands back only a total, so a Director reading "eleven Guides were active" is not reading anybody's conversation.

> **NOTE** · An Explorer does not see the church counted
>
> *Your church at a glance* — Guides, Explorers, waiting for approval, graduated, where people are on the journey — is on the Church screen for Guides and leadership, and not for Explorers. It names nobody and shows no conversation, which is why it was on everyone's screen at first. Safe is not the same as theirs: it is the church looking at itself, and an Explorer opening their church screen was shown a tally of how many people like them there are and how many had "graduated". Community Blogs and the church's notices are what that screen is for.

The board report

Admin has a panel with the four numbers to read out at a board meeting, and a Print button. It names nobody. If a board member wants to know how one particular person is doing, the answer is to ask the Guide walking with them. The app will not show it.

5. Getting it onto a phone

Hope Beacon installs from the browser. There is no app store, no download, and no review process. Once installed it has its own icon, opens without an address bar, and keeps working when the signal does not.

> **IMPORTANT** · iPhone and iPad: only Safari can install
>
> Apple permits only Safari to add an app to the Home Screen. Chrome, Firefox, Edge and Opera on an iPhone cannot do it, and no amount of work on our side can change that. Neither can the browser inside Messenger, Facebook or Instagram.
>
> What we can do is make leaving take one tap instead of three steps, and that is what the app now does.

If you are not in Safari: one tap

Open the app in Chrome, Firefox, Edge, or from a link inside Messenger or Facebook, and it says which browser you are in and offers a single button: **Open this page in Safari**. Tapping it reopens the page you are on, in Safari, with nothing retyped. From there, Share and Add to Home Screen work normally.

This matters most for somebody holding an invitation. The old advice was to switch to Safari, and people did that by opening Safari and typing the address, which loses the invitation link they were on. That is the version of the bug reported as "they switch to Safari and it still does not work". The button carries the exact page across.

> **CAUTION** · Honest about the limits of this
>
> The handoff uses a URL scheme Apple has never documented. Most browsers and most in-app browsers honour it. Some refuse, and when they do, *nothing happens and nothing says why*.
>
> So the written steps stay on the screen underneath the button, always, and there is a Copy link button for the person whose browser refuses both. It has been tested with simulated iPhone browsers in an automated test. It has **not** been tested on a physical iPhone.

iPhone and iPad, by hand

1. Open the church's address **in Safari**. If you are in another browser or inside a chat app, use its ••• menu and choose **Open in Safari**.
2. Tap **Share**, the square with an arrow coming out of it.
3. Scroll down and tap **Add to Home Screen**.
4. Tap **Add**, top right.

> **CAUTION** · If somebody installed before 26 August 2026
>
> What they have is a bookmark, not an app, and it will not convert itself. Older iPhones needed a tag the framework had stopped emitting, so Add to Home Screen produced an icon that opened Safari with the address bar showing. Nothing errored, which is why it was reported as "the install does not work".
>
> The fix is deployed. Those people must **delete the old icon and add it again**.

Every other browser

Settings asks which browser you are in and gives that browser's steps. It guesses from the browser itself and opens on the right one, and the whole list is one tap away if the guess is wrong.

Browser	Where	What to press
Chrome	Android, Windows, Mac, Linux, Chromebook	Phone: ⋮ then **Add to Home screen**. Computer: the install icon at the right-hand end of the address bar.
Microsoft Edge	Android, Windows, Mac	Phone: ••• at the bottom, then **Add to phone**. Computer: the install icon, or ••• then **Apps** then **Install this site as an app**.
Samsung Internet	Samsung phones and tablets	☰ at the bottom right, then **Add page to**, then **Home screen**.
Opera	Android, Windows, Mac	Phone: the menu, then **Add to**, then **Home screen**. Computer: the install icon in the address bar.
Brave	Android, Windows, Mac	Phone: ⋮ then **Add to Home screen**. Computer: the install icon, or ☰ then **Install Hope Beacon**.
Vivaldi	Android, Windows, Mac, Linux	The menu, then **Add to Home screen**. Computer: the install icon.
Firefox	Android only	⋮ then **Add to Home screen**.
Hola, and any other Chromium browser	Android, Windows, Mac	Open the menu and look for **Install**, **Install app** or **Add to Home screen**.
Safari	iPhone, iPad	**Share**, then **Add to Home Screen**. The only one that works on Apple.
Safari	Mac	**File**, then **Add to Dock**.

> **NOTE** · Why the list ends with a catch-all rather than every name
>
> Hola, Kiwi, Yandex, UC, DuckDuckGo and the rest are all built on the same engine as Chrome, so they install the same way and only the wording moves. Naming every browser that exists is a list that is wrong the week after it is written; **Install** or **Add to Home screen** is what to look for in any of them.
>
> Two things decided that the browser is *offered as a choice* rather than detected. **Brave does not put its own name in the identifier a browser sends**, so it cannot be recognised that way at all. And searching that identifier for "Hola" matches `Le Hola`, which is a **phone model**, not a browser: a member on that handset would have been told they were using something they have never installed. Both are checked by a test.

> **GOOD TO KNOW** · Firefox on a computer cannot install web apps
>
> Not a setting, and nothing to turn on. Use Chrome, Edge or Safari on a computer. Firefox on a phone is fine and is in the table above. The app says this plainly rather than offering steps that cannot work.

Updates

Nobody reinstalls. When a new version ships, every open copy notices within seconds and offers to refresh. The app will not reload while a message is half written.

> **GOOD TO KNOW** · An update does not sign anybody out
>
> Signing in is stored in the browser, tied to the database project, and shipping new code does not touch it. Nor does the offline cache being rebuilt, nor the crash recovery, which clears caches only. There is a test that fails the build if that ever stops being true.
>
> **Two things do end every session, and both are in Part 7:** moving to a different database project, and changing the web address.

Something to listen to

There is one player, in two sizes, and they are two views of the same thing rather than two players.

The small one sits in the right-hand column of every room. Shut, it shows what is playing and the volume. The ⋯ at its top right opens it, and it remembers which you chose on that device. Open, it has the same three tabs as the full one.

The full one is the first thing on **My Library**, and it is the only place it appears. It used to sit on My Journey and on a Guide's workspace as well, where it was the largest card on a screen meant to be about somebody's next step.

It is a real player, not a play button. Both sizes have:

- a **progress bar you can drag**, with the time so far on the left and the time remaining on the right;
- **previous** and **next** through whatever is queued;
- **back ten seconds** and **forward ten seconds**, which is what you actually want in a talk;
- **mute**, and a volume slider.

Pressing previous once restarts the track you are on, the way every player people already use behaves. Pressing it again goes back a track.

Video plays too, with a picture. A video's picture appears in whichever player you are looking at, and follows you: start one in the Library, go back to your room, and it keeps playing with its picture in the rail.

Three tabs, in the order most people want them.

- **Vault** is your own music and video, saved on this device. There is a search box over it, because a vault worth having is a vault too long to scroll, and a **Save music or video** button that puts more in. Files stay on the device and are never uploaded.
- **Playlists** are your own, saved on the device and never uploaded: name one, then add whatever is playing to it. A playlist can mix ambience and your own recordings, so rainfall behind a sermon is one list.
- **Ambience** is **made on the device as it plays**. There is no file to download, it costs no data, and it works with no signal. It has no progress bar, because it has no end; the player says so rather than showing a bar that never moves. It comes in **two groups, and the split is the point**: *Calm* (rainfall, distant surf, night wind) is made to sit with while you read, and *White noise* (soft hush, plain hush) is brighter and flatter to cover the people talking around you. They used to be one list of three, and people looking for something restful kept landing on the flattest thing in it and reporting that the sound was unpleasant. Each one now says what it sounds like underneath its name, including the honest warning on the harshest.

Both sizes drive the same element, so starting a track in the Library and walking back to your room keeps it playing.

What somebody listens to while they read is nobody else's business, which is why the vault and the playlists stay on the device rather than in the church's database.

What is in the left column

Home, your own screen, the **Guild Room**, the library, **Publish** and **Cases**. Guides and leadership also have the **Office**. Under **You**: Profile, Mail and Settings.

Six of those rooms have **subrooms** inside them, offered as a row across the top when you open one. See *Rooms and subrooms* in Part 4.

> **IMPORTANT** · On a phone there is no left column
>
> The column appears only on a screen at least 1280 points wide, which is a laptop. On every phone, and on an iPad held upright, the scrolling row of icons under the header **is** the navigation. It is not a shortcut to some of it.
>
> So the two lists have to hold the same rooms, and for a while they did not: Office, Publish and Cases were added to the column and never to the row, and were unreachable on a phone. There is a check that fails the build when they disagree. If you add a room, add it to both.

Tutorial, What's new and Feedback are cards inside Settings, not rows in the column. On a live church that was half true for a while: only the tutorial made the move, so What's new and Feedback existed in the sample-data build and nowhere else. Both are now on a **Help and feedback** card in Settings, on both. They were rows for a while, put there because each had been reported as missing when it was only reachable by scrolling Settings. That fixed the

wrong half: it made the column six entries long for three things somebody uses about once a month, and the column is what people look at all day. The unread mark for a new release sits on Settings itself, so it is still visible from every screen.

On the desk

The right-hand column carries a panel for Guides, Directors and Executive Directors: what is **coming up**, and what is **waiting for you**.

Coming up is the next three meetings, with the day and the time; pressing one opens the conversation it belongs to. Waiting for you is everything somebody is waiting on you for: unread notifications, prayer requests, people awaiting approval, Explorers with no Guide, open safeguarding reports, and Guides asking to be paired with somebody.

Every line goes to the exact card, not the top of a page. *Prayer requests waiting* opens the prayer card itself; *Waiting to be approved* opens Admin already on Approvals rather than on whichever tab you last used; *Guides asking to walk with somebody* opens the Office at that card. The card is marked briefly when you arrive, so you can see which one answered the press. *Unread notifications* is the one exception and it is not a link: the bell is in the header of every screen, so pressing that line opens the bell where you already are.

Lines disappear when the work is done, so an empty panel means an empty desk.

> **NOTE** · This is what "it doesn't go to the feature" was
>
> The lines were links, but they pointed at pages. Arriving at the top of a long screen and having to find the thing you just pressed is the same as not being sent — and pressing an `/admin` line while already on Admin did nothing at all, because the address changed and the screen did not. The desk rail is drawn on the page it links to, so that was the usual case rather than an unlucky one.

Explorers get the study timer here instead. An Explorer has no queue of work, and giving them a "waiting for you" panel would invent one.

> **NOTE** · This panel used to be empty always
>
> It was passed a hard-coded empty list, so every signed-in person was told "Nothing waiting. A good place to be." whatever was actually waiting. It looked like a considered empty state and had never been connected to anything.

Pop-ups on a phone held upright

Every panel that opens over the page (the bell, the account menu, the player's menu, the share sheet, the install card) is measured against the screen the phone actually has, and is pinned inside it.

Three things were wrong, and all three showed up only in portrait:

- **A panel was measured against the wrong screen.** The unit used for "most of the height" is the page's idea of the viewport, which on a phone is the height with the address bar hidden. Held upright, a panel asking for 90% of that was taller than the screen and its bottom was unreachable.

- **A panel anchored to a button near the right edge ran off the side**, because it was positioned from that button rather than clamped to the screen. In landscape there was room and nobody saw it.
- **Anything pinned to the bottom sat under the home indicator** on a modern iPhone, which reserves about 34 points that a fixed offset knows nothing about.

They are all one component now, so a new panel gets the behaviour instead of somebody having to remember it.

Waiting

Opening the app shows one screen while it signs you in: the app's own mark with a halo breathing behind it, the name, and a bar that travels without pretending to know how far along it is.

There were three waiting states and the app showed the plainest of them. The designed one existed and nothing ever drew it; what people saw was a second, flatter screen written inside the live shell, with a lighthouse drawn by hand rather than the app's actual logo. So somebody saw one mark while waiting and a different one for the rest of the session. The church app this grew out of had already found that and fixed it, and this is the same fix.

Anywhere the app is fetching something smaller, it shows the same mark turning with a word for what it is waiting on, rather than the word "Loading" on its own or nothing at all. A screen that is still fetching and a screen that has finished and found nothing used to look identical, so people pressed the button again.

A device set to reduce motion gets the same message without the spin.

Notifications

The bell in the header holds both the list and its switch. Alerts in the app are **on by default**.

Pressing one opens what it is about. A safeguarding notice opens the Safeguarding room for a Director and the Cases room for anybody who has been called to one; a prayer notice opens the prayer card; an approval opens Approvals. It also marks the notice read, which it always did — before tonight that was *all* it did, so the bold went away and the person was left on whatever screen they were already on with no idea where to go.

They pop up on the device, not only in the list. Tap **Turn on device alerts**, allow it when the browser asks, and one appears straight away so you can see it worked. After that, anything new reaches the notification tray on your phone or computer while Beacon is open in a tab or installed, and tapping it opens the app on the right screen.

At most three pop up at once. Somebody coming back to eleven unread things needs to be told, not buried, so the rest stay on the badge.

You are told when you arrive, not only while you are watching. Signing in, or coming back to the app after being offline, checks what is waiting and says so. Before this, an alert only ever appeared if you happened to be looking at the app the moment it was created, so somebody who closed their laptop on Friday and opened it on Sunday was greeted by a silent screen with a number on a bell they had no reason to look at.

If several things arrived while you were away, you get one line saying how many rather than a stack of pop-ups.

> **NOTE** · What is not built
>
> A notification when Beacon is **completely closed** needs a push service and a signing key held on a server, which this church does not have set up. The app is ready for it: the service worker already handles a push and a tap. Until those keys exist, alerts arrive while Beacon is open in a tab or running as an installed app, which for a phone with the icon on the home screen is most of the time.

Device alerts are the browser's to grant: the panel offers to ask once, and if a browser has already refused it says where to change that rather than offering a button that would do nothing. Once granted, the panel says so, because the only other way to know it worked was to wait for one.

6. Email, end to end

Two providers, deliberately, because they fail in different ways and one must not be able to take the other down.

What	Sent by	Why
Invitations	**Brevo**	Three different messages, one per role, composed in the code and kept under version control.
Password resets	**Supabase**, over Brevo's SMTP	Only the auth system can mint a recovery link, so this one cannot move.

Why not Supabase for invitations too

Supabase Auth has exactly one "Invite user" template with no way to branch on a role. The moment three roles needed three different invitations, that template could no longer do the job. There is also a hard ceiling: the built-in mailer sends **two emails an hour for the whole project**, which one Director inviting three people on a Sunday afternoon would exhaust.

Setting up Brevo

Step 1. Verify your sending domain. In Brevo, go to *Senders, Domains & Dedicated IPs* and add your domain. Brevo gives you DNS records to publish; add them at whoever sells you the domain and wait for Brevo to show the domain as authenticated.

Step 2. Create an API key. *SMTP & API → API keys → Generate a new API key*. It begins with `xkeysib-`. Copy it once; Brevo will not show it again.

Step 3. Create an SMTP key as well, on the same page. This is a different key for a different job, and the password reset needs it.

Step 4. Check the IP restriction on both. Brevo can limit a key to named IP addresses, and it is two separate switches: one for API keys, one for SMTP keys. Our server has no fixed address, so a key restricted this way is refused every time and Brevo's log shows nothing at all.

Step 5. Store the API key in Supabase, never in the website's settings. See Part 8 for exactly where.

> **CAUTION** · Two Brevo traps that cost a morning each
>
> **Not every key works with the API.** Keys created for other Brevo integrations are a different type, and the sending endpoint answers "Key not found" for them, which reads as a wrong key rather than a wrong kind. Create the key from *SMTP & API → API keys* and nowhere else.
>
> **IP restriction is on by default in some accounts.** Turning it off is a real reduction in protection, and it is the owner's call. The compensating control is that the key lives in the database where only the server can read it, and rotating it takes under a minute.

Setting up the password reset

In Supabase: *Project Settings → Authentication → SMTP Settings*. Enable custom SMTP and enter Brevo's host, port `587`, your Brevo login and the **SMTP key** as the password. Set the sender to an address on the domain you verified. This also lifts the two-an-hour ceiling for everything Supabase sends.

How an invitation is actually sent

Worth understanding, because almost every email failure has been a misunderstanding of this.

1. A Director presses Send. The request reaches a small server function, the only piece of the system holding the key that can bypass the security rules.
2. It refuses if the address already belongs to a member who has finished signing up.
3. It creates or refreshes the one invitation row for that address.
4. It mints a one-time link, composes the message for that role, and hands it to Brevo.
5. If Brevo will not take it, and only then, it produces a link the Director can pass on by hand, and says why the email did not go.

> **IMPORTANT** · The rule that broke every invitation for a week
>
> An account has **one slot** for an invitation link, not a collection. Minting a second link overwrites the first, and the first stops working immediately. The function used to mint a spare link after sending the real one, which quietly destroyed the link that had just gone into somebody's inbox. Every invitation arrived dead and the error message said the link had expired.
>
> Never mint a link after a send. Anything that calls the mint function must do it before the send, or only when the send has failed.

What the invitation actually says now

The message leads with **how to install Hope Beacon**, with the steps for Safari on an iPhone or iPad and the steps for any other browser, and the **Accept your invitation** link sits at the foot of it rather than at the top.

That order is deliberate. The link is one-time: opening it, glancing at a sign-in screen on a browser they will not keep using, and closing it again is how somebody burns their invitation before they have the app. Reading how to install first, then accepting, is the path that works.

Each role is also told which room to open first, so the first screen after joining is not a guess.

Changing the wording of an invitation

The three messages live in the repository at `supabase/functions/invite/email.ts`. Edit them there, in plain TypeScript, and redeploy the function. A test renders all three and checks the link appears twice, the church name is escaped, and no placeholder survives into the message.

Brevo templates are supported as an alternative but not recommended: they put the words a congregation reads behind a dashboard with no version control, and somebody must build three by hand before a single invitation can go out.

7. Moving to a new project

This is the part to read twice. Moving the app means moving three separate things, and they have different consequences for the people already using it.

What moves	Effect on people already using it
The code (a new repository, a new deploy)	None. Nobody is signed out and nothing is reinstalled.
The database (a new Supabase project)	**Everyone is signed out, and their accounts do not come with it** unless you deliberately carry them over.
The web address (a new domain)	**Everyone is signed out, and every installed icon is stranded for good** on a copy that can never update.

> **IMPORTANT** · The address is the one you cannot undo
>
> A browser identifies an installed app by its web address. Change it and every phone that installed the old one keeps a copy that can never receive another update, and no amount of work on our side reaches it. The only fix is to ask every person to delete the icon and add it again.
>
> So: decide the final address *before* more people install, or accept that one day you will send that message to everybody. There is no third option. If you do move, announce it before the switch, not after.

Why a new database signs everybody out

Being signed in is one entry stored in the browser, and its name contains the database project's own identifier. A different project means a different name, so the browser looks for the old one, finds nothing, and shows the sign-in screen. The accounts themselves live inside the old project and do not travel with the code.

Two honest ways forward. Choose deliberately.

	A. Start clean	B. Carry the accounts over
What you do	Run the migrations on the new project and invite everybody again.	Copy the database, including the authentication tables, into the new project.
Passwords	Everybody sets a new one.	Survive the move.
Signed out	Yes.	Yes, unavoidably.
Risk	Low. Nothing to go subtly wrong.	Higher. Needs direct database access and careful ordering.
Right when	A demo, a pilot, or a church small enough to re-invite in an afternoon.	A congregation with real history worth keeping.

The migration checklist

In this order. Each step is checkable, and a step that cannot be checked has not been done.

Step 1. Create the new Supabase project. Note its URL and its publishable (anon) key from *Project Settings → API*. The anon key is not a secret and ships to every browser by design; the service role key never leaves the server.

Step 2. Run every migration, in filename order, from `supabase/migrations/`. They build on one another, so the order is not optional. Migration `0001` through `0035` plus the dated one. Paste each into the SQL editor, or use the Supabase CLI.

Check: the `profiles`, `pairings`, `invites` and `app_settings` tables exist, and row level security is on for all of them.

Step 3. Deploy the invite function. Deploy `supabase/functions/invite` to the new project. It is the only piece holding the service role key, and it is what sends every invitation.

Check: the function appears in *Edge Functions* and its version is the one you just deployed, not an older one that happened to be there.

Step 4. Put the Brevo key in `app_settings`. In the SQL editor:

```
insert into app_settings (key, value) values
  ('BREVO_API_KEY',     'xkeysib-your-key-here'),
  ('BREVO_SENDER',      'hello@your-domain.org'),
  ('BREVO_SENDER_NAME', 'Your Church')
on conflict (key) do update set value = excluded.value;
```

That table has row level security on and no policy granting anybody access, so only the server can read it. It is not in the repository and never will be.

Step 5. Set the sign-in redirect. *Authentication → URL Configuration*. Set *Site URL* to your address, and add `https://your-address/join` to the redirect allow list. Get this wrong and invitations arrive but land nowhere.

Step 6. Turn on custom SMTP for password resets, as in Part 6.

Step 7. Point the website at the new project. On the host, set `NEXT_PUBLIC_SUPABASE_URL` and `NEXT_PUBLIC_SUPABASE_ANON_KEY`, then **redeploy**. Changing a setting alone does nothing: these are read at build time, so a saved setting with no redeploy leaves the old project connected.

Step 8. Create the first account by hand. There is no public sign-up, which leaves the first account a chicken and egg problem. In *Authentication*, add a user with your email and a password. Then in the SQL editor find that person in `profiles`, set their role to `executive` and their approval to true.

Step 9. Prove the rules before real names go in. Sign in as somebody with the least access, an Explorer, and try to reach what they should not: another person's conversation, the member list, the admin screens. The rules are enforced by the database rather than by the screens, so this is a real test. `docs/examples/prove-the-rules.sql` does the same thing faster.

Step 10. Send one real invitation to yourself and complete it end to end: receive it, choose a password, get approved, land in the app. Only then invite anybody else.

> **GOOD TO KNOW** · What "working the same" means, concretely
>
> After step 10, all of this should be true on the new project: an invitation arrives within a minute; the three roles get three different messages; a password reset arrives; an Explorer cannot see another Explorer; a Guide cannot take a sixth Explorer; deleting an account frees the address; and the app installs on an iPhone from Safari. If any one of those is false, stop and fix it before the next step rather than after.

8. Every setting, in one place

On the website host

Name	What it is	Required
`NEXT_PUBLIC_SUPABASE_URL`	Your project's address. Public by design.	Yes
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	The publishable key. Public by design; the security rules are what protect the data.	Yes
`CANONICAL_HOST`	The address the app treats as its real home, comma separated if there is more than one. What lets the app warn somebody who installed from a temporary address.	Recommended
`NEXT_PUBLIC_VAPID_PUBLIC_KEY`	Only if you add push notifications later.	No

> **IMPORTANT** · Never
>
> The service role key must never appear on the website host, and never in anything whose name begins `NEXT_PUBLIC_`. That key bypasses every security rule in the database. It belongs in the invite function and nowhere else.

In the database, in `app_settings`

Key	What it is
`BREVO_API_KEY`	The API key. Without it, invitations fall back to Supabase and its two-an-hour ceiling.
`BREVO_SENDER`	The address invitations come from. Must be on a domain Brevo has verified.
`BREVO_SENDER_NAME`	The name people see, for example your church's name.
`SITE_URL`	Where invitation links point, if it differs from the site's own idea of itself.
`BREVO_INVITE_TEMPLATE_ID_DS`	Optional. A Brevo template for Explorers, instead of the built-in message.
`BREVO_INVITE_TEMPLATE_ID_DM`	Optional. The same, for Guides.
`BREVO_INVITE_TEMPLATE_ID_ADMIN`	Optional. The same, for Directors.
`BREVO_INVITE_TEMPLATE_ID`	Optional. One template for every role, used only when no per-role template is set.

Any of these may instead be set as a secret on the invite function; the function checks its own secrets first and the table second. The table is easier to change without a redeploy.

DNS, at whoever sells you the domain

Purpose	What to add
The website	The records your host gives you when you add the domain to the project.
Email authentication	The DKIM and SPF records Brevo gives you when you add your sending domain. Without them, invitations land in spam.
DMARC	Optional but worth it once the two above are verified.

9. The database and its rules

Every privacy promise this app makes is kept by the database, not by the screens. That distinction is the whole security design: a screen can be bypassed by anybody who opens the developer tools, and a database rule cannot.

The rules that must never break

Rule	Kept by
Nobody can change their own role.	A trigger that rejects the change, and a saved profile that always writes back the role it read.
An Explorer sees only themselves.	Row level security on every table, scoped by church and by pairing.
A Guide sees only the Explorers paired with them.	The same.
A Guide carries at most five Explorers.	A trigger that counts, because a limit across rows cannot be a constraint.
The Head Executive Director cannot be removed.	The discipline check, refused before anything happens.
A removal is always recorded.	A log written before the deletion, which outlives the person it describes.
A change to somebody's details is visible to their Guide.	An append-only table with no write policy at all, filled by a trigger.
Nothing new is readable by a signed-out visitor.	A check that fails the build if any table grants anything to anonymous.
Only an approved, unsuspended member can publish a post.	The write rule on the posts table, which also pins the author to whoever is signed in and the church to their own.
A Director may take down any post in their church.	The delete rule, scoped to churches that Director actually leads.
Counting messages never exposes one.	The counting runs inside the database and returns totals. No message, and no name, ever leaves it.
Only members of a guild can read or write its board.	A definer function that checks membership and refuses anybody else. The tables themselves grant nothing to anybody.
A guild board never publishes who is in the guild.	The function returns *You*, *A Guide* or *A fellow Explorer*, and no identifier at all.
Reporting a guild post never reveals its author.	The browser sends a post, not a person. The author is resolved inside the database and never returned.
A reported post survives being deleted.	Its text is copied into the report when the report is made.
Only leadership of that church can take a post down.	`leads_church`, which also covers an Executive Director set over several congregations.

Rule	Kept by
The security audit is leadership only.	A definer function that refuses anybody else. The table has row-level security on and every grant revoked, so the function is the only way in.
An unapproved account cannot approve itself.	The same trigger that pins roles. The one exception it allows, claiming an invitation, cannot set approval, only church and role, and only from an unexpired invitation addressed to that account's own email.

The blog error, and what it actually was

Worth writing down, because it was diagnosed wrongly twice and each wrong fix looked plausible.

Publishing a post failed with `new row violates row-level security policy`, which reads exactly like a refusal to write. It was not. The app saves a post and asks for its id back in one statement, and the database applies the **read** rule to the row it is about to hand back. The read rule called a helper that looks the post up by its id, and that helper runs on a snapshot of the database taken before the row existed. So it looked for the post, could not find it, and refused to return to the author the very row it had just accepted from them.

The fix is to answer the author's own case from the row itself rather than by looking it up: the post is yours if its author is you, which needs no lookup and is true for a row nothing can see yet.

Two things made this hard to see. The message names the write, and the same insert works perfectly if you do not ask for the id back. It was proved by doing exactly that, and by widening the write rule to allow everything and watching it fail anyway.

Deleting an account, and why it used to fail

Worth stating because it is the kind of bug that hides for months. The `profiles` table hangs off the authentication table, and deleting a profile does *not* delete the account behind it. A cascade only runs one way.

So a removed person kept a working login that resolved to nothing, and their email address could never be invited again, because the check that refuses a duplicate invitation looks at the authentication table, which still held their row. Only deleting there frees the address, and that is what the app now does everywhere a member can be removed.

Migrations are named by time now

The early files are numbered `0001` to `0049`. Everything since is named `YYYYMMDDHHMMSS_`, and new ones must be too.

This is not tidiness. Migrations run in filename order, and `0050_` sorts **before** `20260829…`. A migration numbered today would run before the tables it depends on. On a machine where the database already exists nothing at all would happen; a database built from scratch would fail. There is a check that refuses a `00NN_` file above the closed series.

Two more things about migrations, both learned here:

- **Never edit one that has already been applied.** Editing the file changes nothing in the database and quietly makes a fresh environment differ from production. Add a corrective migration instead. (Editing one that failed on a syntax error and never ran is fine, because nothing has it yet.)

- **The version recorded in the database does not match the filename.** Migrations applied through the tooling are stamped with the time they were applied. Do not read the two as if they line up; check whether the objects exist instead.

Photographs, and what they carry

A photo from a phone is two to four megabytes, and a conversation shows it a few hundred pixels wide. Every one of those megabytes is paid for twice, once to store and again on **every single view**, because a private file is fetched through a fresh signed link that no cache will keep.

So a photo is made smaller before it is sent: sixteen hundred pixels on its longest edge, which nobody can tell apart on the screen it is read on. Measured in a real browser on a real 4032-pixel photograph: **82% smaller**. Fifteen of the sixteen files a real church had sent each other were photographs averaging 2.3 MB, so this is most of the storage bill and most of the traffic.

> **IMPORTANT** · It also removes where the picture was taken
>
> A phone writes the exact coordinates into a photograph, usually somebody's home. Sending a picture of a Bible page to a Guide should not tell them where you live. Re-drawing the image keeps the pixels and drops every tag, so the privacy fix and the saving are the same line of code. The conversation says so above the message box, and so does the privacy notice.

A photograph is shown, not named. A picture sent into a conversation draws as the picture, tappable for the full size; a voice note draws as a player. Anything else is a filename, which is right for a study sheet. The sample side had done this for months and the live conversation had not, so somebody who learned the app in the tutorial signed in, sent their Guide a photo, and got `20260901_110714.jpg`.

Pictures further up a thread are not fetched until they are scrolled to, and a voice note downloads nothing until it is played. That is not politeness: a private file is fetched through a signed link no cache keeps, so a conversation with thirty photographs in it would otherwise download all thirty to show today's message.

> **NOTE** · Why there is no video
>
> Video cannot be sent today. A minute of it is about four hundred shrunk photographs, and it is paid for again on every view. Put it on YouTube and share the link; the composer says so.
>
> **This is under review and the cost has been worked out.** At this church's size video turns out to be affordable, and it stops being affordable at about ten churches sharing one instance. What it costs (https://github.com/klydo131/open-sentry-beacon/blob/main/docs/./WHAT-IT-COSTS.md) has the numbers, the three arguments against that are not about money, and the recommendation: allow it with a short cap, and measure for a month before widening it.

Files in the library never reach a server at all. The library holds links; a file saved under *On this device* is passed from one phone to the other through the phone's own share sheet. Files in a **conversation** do reach the server, because the two people are rarely holding their phones at the same moment and the file has to wait somewhere. That is the honest line between the two.

Backups

Two scheduled jobs live in the repository. Both are free on a public repository, and both need their secrets set before they do anything:

- **Keep awake** pings the database daily, so a free project is never paused for inactivity.
- **Backup** takes an encrypted copy weekly.

> **IMPORTANT** · Two things about backups
>
> **A backup nobody has restored is a rumour.** Restore one into a scratch project once, before you need it. The job has never been proved by a real restore.
>
> **Never let an unencrypted dump reach the repository.** Files attached to a job on a public repository can be downloaded by anyone on the internet. Encrypt before upload, or do not produce the file.

The signed-out role holds nothing

Supabase gives the anonymous role every privilege on every new table, and Row Level Security is what takes it back. An audit on 1 September found thirteen policies written with no `TO` clause, which means they applied to everybody including a visitor who has not signed in, and about twenty tables still carrying the default grant.

Nothing was leaking. Every table was probed as that role and every one returned nothing, because each of those policies compares something against `auth.uid()`, which is empty when nobody is signed in. But that is safety by arithmetic rather than by design: the anonymous key ships inside the JavaScript that every visitor downloads, and the next policy written as `using (is_published)` would have opened its table to the internet while looking perfectly reasonable in review.

So the signed-out role now holds no table privilege at all, every policy names the roles it is for, and an event trigger takes the grants off any table created from now on, in the same way one already locks down new functions. A new table arrives with Row Level Security on and no grants.

The bytes get the same boundary as the rows

Two storage rules matched on the folder name alone, so any signed-in account could list and download every lesson file and every avatar in the instance, whatever church it belonged to. The table describing those files was correctly scoped by church the whole time; only the files were not. Both are now scoped through `can_access_church`, which still lets an Executive Director see the churches they oversee.

> **IMPORTANT** · A deleted account and its pictures
>
> Deleting a row from `storage.objects` does **not** delete the image. Supabase refuses direct deletes from that table on purpose, because the row is only metadata and removing it strands the file rather than removing it. A first attempt at this shipped as a database trigger, which could never have worked and reported success anyway. Clearing somebody's files has to go through the Storage API, from the app, **before** the account is deleted, because afterwards nothing can say which church the files belonged to.

10. What it costs

Worked out from the live systems rather than estimated, and it produced two surprises worth stating here.

This app is not on a free plan. The Supabase organisation is on **Pro**, and there are **two projects** on it: the live one, and the predecessor with nine accounts that nobody uses. That is about **$35 a month, roughly ₱2,030**, against a stated budget of ₱2,000. Pausing the old project brings it to about **₱1,450**, and is the single cleanest saving available.

Every document in this repository said "free plan", including this handbook, and that error was informing decisions.

Storage will not be a problem this decade. Sixteen days in, with 63 members, the database is 16 MB of an 8 GB allowance and the files are 34 MB of 100 GB. Traffic is the meter that moves, because a private file is downloaded again on every view.

The bill does not multiply as churches join, because the plan is per organisation and the app is already multi-tenant. Ten churches on one instance is about ₱145 each. Ten separate projects would be ten times the compute for the same work.

docs/WHAT-IT-COSTS.md (https://github.com/klydo131/open-sentry-beacon/blob/main/docs/./WHAT-IT-COSTS.md) has the measured figures, the video question costed both ways, and the list of what could not be seen from here. Chief among it is the Vercel account that actually serves the site, which is the one number that could still put the church over budget.

11. Data protection

Somebody will eventually ask what the law requires of a church running this, and the answer starts with one fact.

This app holds sensitive personal information. Under the Philippine Data Privacy Act (RA 10173) a person's **age**, **marital status** and **religious affiliation** are all sensitive, and this app records a birthday, a life status, and the whole of somebody's participation in a church. Membership is a religious affiliation, so every row is sensitive whether the column looks it or not.

That raises the standard in four ways: consent has to be **express** rather than implied, a **Data Protection Officer** is expected, the church must **register with the National Privacy Commission** once it passes a thousand members, and the penalties for getting it wrong are higher.

docs/DATA-PROTECTION.md (https://github.com/klydo131/open-sentry-beacon/blob/main/docs/./DATA-PROTECTION.md) is the map: every table that holds anything about a person, why it is there, who can read it, how long it stays, and which company holds it. It also lists what is still missing, and the list is the point of the document.

Asking for a copy of your own information

Anybody signed in can download everything the app holds about them, from **Profile → A copy of your information**. No reason is needed and nobody has to approve it. The file is JSON, which is the form both laws ask for so it can be carried somewhere else.

It is built from what that person could already read. There is no privileged path assembling "everything about member X", because that is one mistake away from handing somebody a stranger's conversation, and the file would look the same either way. Proved against the live database: reading every message as one Explorer returned three, all

theirs, and none from a conversation they are not in.

Three things are deliberately left out, and the file says so on its front page: a safeguarding report about them, a Guide's private notes, and the record of approvals and removals. A report names whoever raised it, and this app promises that person the one they reported is never told; handing it over would break that promise and could put somebody at risk. Both laws allow an access request to be limited where answering it would identify another person who has not agreed.

The file names each exclusion, gives the reason, and says to write to the Data Protection Officer, who can weigh a particular case. That is the difference between an omission somebody was told about and one they were not.

The two largest gaps today:

1. **The privacy notice is a draft.** `/privacy` in the app is written and reachable from Settings, with the church's name, the DPO's contact, the hosting region and the retention periods marked as blanks. They cannot be guessed, and a notice with a plausible wrong name on it is worse than one that admits it is unfinished.
2. **Nobody has decided the retention periods.** Everything except the 30-day library record is kept because nothing deletes it, not because a period was chosen. The privacy notice has a blank waiting for the answer.

> **CAUTION** · This is the factual half, not a legal opinion
>
> An engineer can say what the app collects and who can see it. Whether that satisfies a regulator is a question for a lawyer, and neither the document nor this section is legal advice. What they do is give a lawyer the map they cannot produce themselves.

12. When something breaks

What you see	What it usually is	What to do
"This invitation link has expired or has already been used"	A newer invitation was sent to the same address, which switches off the older one. Or somebody already opened it.	Ask them to use the newest email. If unsure, press Re-send and tell them to use only what arrives after that.
The invitation email arrives empty	A template using a field the mail system cannot resolve. It abandons the whole message and sends a blank one, and nothing anywhere reports an error.	In a Supabase template, keep to `{{ .SiteURL }}` and `{{ .TokenHash }}`. Never `{{ .ConfirmationURL }}`: it points at Supabase's verify endpoint, which spends the token on any GET, so a mail scanner burns the link before the reader taps it.
No invitation arrives at all, and Brevo's log is empty	The key was refused before a send was recorded. Almost always the IP restriction, occasionally a key of the wrong type.	Check the IP setting on *both* API and SMTP keys. Then confirm the key was made under *SMTP & API → API keys*.
"one message per address per minute"	Working as intended. A second message to one address inside a minute is held back.	Wait the number of seconds shown, then press Send once.
Two invitations arrive and the second looks blank	Gmail collapses a later message that resembles an earlier one in the same thread behind "Show quoted text".	Expand the quoted text. The three roles now have three different subject lines, which prevents most of this.
"already has a Hope Beacon account"	That address finished a sign-up before, possibly at another church.	If they are in your church, change their role from the member list. If they have genuinely left, delete the account, which frees the address.
The install button does nothing on an iPhone	Not Safari. Chrome, Firefox, Edge and in-app browsers cannot install on iOS.	Tap **Open this page in Safari** on the card, then Share, then Add to Home Screen. If that button does nothing, the browser refused the handoff: use its ••• menu instead.
The icon opens Safari with an address bar	What was added is a bookmark from before the fix.	Delete the icon and add it again from Safari.
"This copy can never update"	It was installed from a temporary preview address.	Open the real address, install from there, then delete the old icon.
A setting was changed and nothing happened	The two settings beginning `NEXT_PUBLIC_` are read when the site is built.	Redeploy. Saving alone changes nothing.
Everybody was signed out at once	The database project changed, the web address changed, or the project's signing secret was rotated. A code deploy does not do this.	See Part 7. If the address changed, people must reinstall as well.

What you see	What it usually is	What to do
"Your account is not ready. JWT expired."	Fixed on 26 August 2026. A sign-in lasts one hour unless the app trades its refresh token for a new one, and nothing did, so everybody was signed out an hour after signing in.	Nothing to do. A session now lasts until somebody signs out, and coming back to the app re-checks it.
A stage was advanced by mistake	Advance is one tap.	**Undo, step back** beside it. Recorded as a correction, and the Explorer never sees their stage either way.
A Guide cannot take another Explorer	They are at the church's limit, five by default.	Pair with another Guide, recruit one, or raise the limit in Church settings. Do not raise it to solve a shortage of Guides.
The whole left column is invisible on a dark theme	Fixed on 26 August 2026. The page background was not being themed, so light text landed on a light page.	Nothing to do.
A Guide cannot be given another Explorer	They already have five. The database refuses a sixth.	Pair with a different Guide, or recruit one. Do not raise the cap to solve a shortage of Guides.
"permission denied for table *something*"	**Not** a permissions rule that needs loosening. A rule that refuses somebody shows them nothing; it does not produce this message. This one means the request reached the database with nobody signed in.	Sign out and back in. If it keeps happening, it is the session, not the rule. See the row below.
Signed out after switching tabs, or after opening the app on a second device	Fixed on 28 August 2026. Two copies of the app raced to renew the same session, the loser was told no, and it threw the good session away rather than looking again.	Nothing to do. Signing in now lasts until somebody signs out on that device.
A blank box where an icon should be, on an Android phone	Fixed on 28 August 2026, and again on 31 August. A character that looks like an emoji but comes from a symbol block is drawn only if the phone's font happens to include it. Apple's does; Android's does not.	Nothing to do. Controls are drawn as pictures now, and a check refuses the characters.
A pop-up runs off the bottom or the side, but only in portrait	Fixed on 28 August 2026.	Nothing to do.
An invitation was accepted but the person never appeared in Approvals	Fixed on 29 August 2026. Somebody who already had an account got a recovery link instead of an invitation link, and a recovery link does not carry the church and role across. They existed with no church, visible to nobody.	Nothing to do for new ones. Anybody already stuck in that state was repaired when the fix was applied.

What you see	What it usually is	What to do
"You do not have permission to do that" when adding to the library	Fixed on 1 September 2026, and it was never about permission. The app saved the resource and asked the database for it back in the same breath, and the read rule could not recognise a row that did not exist a moment ago. It is the same fault as the blog error in Part 9.	Nothing to do. Anybody in the church can add a link now, Explorers included.
A card I used to scroll to has disappeared	Nothing was removed. Six rooms now open in folders, and the card is in one of them: the row of choices is across the top of the room.	Tap the folder it belongs to. The room remembers your choice, so it will open there next time.
A link took me to a room but not to the card I pressed for	Fixed on 31 August 2026. Links that pointed at a card by name were pointing at something a folder might not be drawing.	Nothing to do. Those links now name the folder as well, and old ones are translated.
A study cannot be written on a phone or an iPad	Fixed on 28 August 2026. The Office was reachable only from the left column, which does not exist below laptop width.	Nothing to do. Office, Publish and Cases are in the header row on every size.

Where the code lives

Written down because a church that cannot hand this to somebody else owns a liability rather than an app.

Looking for	Open
The signed-out door: home, sign-in, sign-up, joining by invitation	`components/live/DoorPages.tsx`
The Director and Executive Director screen	`components/live/AdminPage.tsx`
A Guide's roster, and one Explorer's page	`components/live/GuidePages.tsx`
An Explorer's own journey	`components/live/ExplorerPage.tsx`
The conversation, and the small parts screens share	`components/live/shared.tsx`
Everything that talks to the database	`lib/live/data.ts`
The rules that keep the promises	`supabase/migrations/`

All of those live screens were one file of three thousand lines called `LiveCorePages.tsx` until they were split by screen. That file still exists and re-exports them, so nothing that imported it had to change; new code should import from the file that holds the screen.

> **NOTE** · Two checks went quiet during that split, and that is the lesson
>
> They read `components/LiveCorePages.tsx` by name, so when the screens moved, ten assertions stopped testing anything while still reporting nothing wrong. One of them was a safeguarding placement check. They read every live screen now, so the next move cannot switch them off. **A check pinned to a file name is a check a refactor can silently delete.**

13. For an AI tool continuing this

Read this section before making a change. It states what is true, what must stay true, and the mistakes already made here so they are not made twice.

> **IMPORTANT** · `AGENTS.md` in the repository root is the working brief
>
> It is longer than this section and it is the one to open first: the two halves of the app, the product rules that outrank a request, how authorisation is arranged, migrations, the verify gate, the phone rules, and the protocol for two agents sharing one branch. This section is the summary; that file is the map. `CLAUDE.md` points at the same place.

The shape of it

- Next.js App Router, TypeScript, Tailwind. Supabase for database and authentication. One edge function, `invite`, holding the only service role key.
- The app runs with **no backend at all**, on sample data in the browser. That is not a fallback, it is a supported mode with tests that fail if it breaks. Never write code that requires the database to exist.
- The security model lives in `supabase/migrations/`. Contracts evolve by adding a migration, never by editing one that has already been applied. **New files are named `YYYYMMDDHHMMSS_`; the `00NN_` series is closed at 0049 and a new number in it would sort first and run before the tables it needs.**
- Anything privileged is a `security definer` function in the `private` schema doing its own check, with a thin `public` wrapper, every grant revoked from `anon`, and the table itself carrying row-level security **and** no grants at all. The function is the only way in.

Invariants you must not break

1. No screen may let anybody set their own role. There is no `setMyRole`, and saving a profile always writes back the role it read.
2. The service role key never reaches the browser and never appears in a variable named `NEXT_PUBLIC_*`.
3. Never mint an invitation link after sending one. An account has one slot and the second mint destroys the first.
4. Removing a member goes through `remove_member_by_leader`. Deleting the profile row alone leaves the account behind and locks the address out for good.
5. Nothing in the update path may clear the browser's stored session.
6. Text a member reads carries no em dashes, calls a Guide a Guide, and does not reach for the cadences a machine reaches for.
7. `select('*')` never appears in `lib/live/data.ts`. The column list is the access control: it is what stands between a birthday and an Explorer's browser, and the return type must have no field for what you did not ask for.
8. Every room where one person can be hurt by another carries all three: a way to report it on the same screen, somebody notified by name whose job it is to look, and a record that outlives the person. Reports have no delete policy at all, deliberately.
9. A control drawn from Miscellaneous Technical or Geometric Shapes is a blank box on Android. Controls are inline SVG in `components/Glyph.tsx`. Emoji are fine; those blocks are not.

10. `dvh` for anything measured against a phone's screen, with a `vh` line beneath it for old browsers, and `env(safe-area-inset-bottom)` on anything pinned to the bottom.
11. A panel lives in exactly one subroom, and the first subroom in the list is what the room is for, because that is the one that opens when nothing is remembered. Never move an Explorer's report control out of the folder holding the conversation, and never put a Guide's church notices inside a folder.
12. One full-screen waiting state, `BeaconSplash`, drawing the app's own mark. A second one written next to the screen that needs it is how this app ended up with three.

Prove it before you claim it

```
npm run verify       # 37 checks: types, build, security, copy, email, install,
                     # phones, sessions, safeguarding
npm run build        # must pass before anything is pushed
```

CI runs the same command on **Ubuntu, macOS and Windows**. A green run on Linux alone is not a green build: one suite here read files with a Unix `find` and quietly checked nothing at all on Windows, and another could not start a process because `npx` is `npx.cmd` there.

> **CAUTION** · A test that passes first time has proved nothing
>
> Every check here was written alongside a negative control: break the thing deliberately, watch the test fail, then restore it. Two checks in this repository passed cleanly over the exact bug they existed to catch, and only the negative control found that out. One reported "all OK" when it had not been able to look at anything at all.
>
> If you add a test, break the code and watch it fail before you believe it.

Mistakes already made here

- **Quoting a count without fetching first.** A stale local copy produced a number four times too large, and it reached a decision.
- **Presenting a blocked network as a design choice.** If something could not be done, say that plainly, then give the reasoning for the fallback separately.
- **Calling unverified work verified.** Pushing is not deploying, and deploying is not observing. Say which of the three happened.
- **Grepping the output of a script for "FAIL" only.** A script that crashed before printing anything looked exactly like a pass.
- **Writing a rule as a list of the cases that existed that day.** The destructive-button check held eight exact button labels, so the ninth was never looked at; its label reader allowed only letters and spaces, so every confirmation that names a person was invisible to it, and those are exactly the presses that carry out a removal. Match the thing, not the list.
- **Shipping a room before asking who is protected in it.** The guild board went out with no report route, no leadership visibility and no way for anyone but the author to delete a post, in a room that includes children. Nothing about it looked wrong on screen.
- **Trusting a filename.** The version a migration is recorded under in the database is not the name of the file it came from, and a `00NN_` name added today sorts before every timestamped one.

- **Editing a migration that had already run.** It changes nothing in the database and makes a fresh environment differ from production. Add a corrective migration.

14. What is not finished

Stated plainly, because a plan that hides its gaps is worse than no plan.

Item	Status
Whether the latest deploy is live	Unverified from here, and it has been unverified for every push. The sandbox cannot reach the site or the hosting dashboard. Needs a person with a browser: open `/version.json` on a phone and check the date.
The iPhone install fix on real Safari	The missing tag is confirmed in the built page. It has not been tested on a physical iPhone.
A restored backup	The job runs and its failure paths are tested. No restore has ever been performed.
Non-English wording	Eleven translations still use the old words for Explorer and Guide. Whether those names translate at all is a decision per language.
The guided tutorial	**Fixed.** The spotlight really was landing on nothing, on a phone, for the Guide's and the Explorer's walks: the panel reserved its clear space on the wrong side, a placement that left the target off screen still counted as done, and the ring could be drawn past the right edge of a 412px screen. All four walks now finish at 393, 412, 768 and 1280 wide.
The tutorial at 375px	The Executive Director's walk still mis-points at one step, and only at iPhone SE width. Director, Guide and Explorer pass at every size.
Three photographs of deleted people	Three avatars belong to accounts that no longer exist. No rule can reach them, so no user can see them, and no rule can delete them either. They need removing from the Storage dashboard by hand.
Creating a new church without a developer	Possible in the database, not yet possible from a screen.
Bulk invitations	**Built**: pasting a list, and dragging a spreadsheet onto the screen. Suggested pairing after a batch is the part still to do.
Safari and iOS behaviour	Checked at iPhone sizes in Chromium, which is not WebKit. WebKit itself is covered by `safari.yml`, which runs every suite on a real macOS machine on each push. Nothing in this app has been seen running on a physical iPhone.
A picture on most Guides' profiles	Almost none have set one, so the card meant to show an Explorer a real person falls back to initials. The app asks them; somebody has to follow it up.
Guild boards in use	The room, the report route and the take-down are built and were proved against the live database. Nothing has yet been posted on one by a real member.
The one-tap Safari handoff on a real device	Tested against simulated iPhone browsers. Never run on a physical iPhone.
Publishing a post, from a real sign-in	The rules were proved against the live database as a real Explorer, Guide, Director and Executive Director, including that a draft never reaches anybody else. It has not been done through the app by a person with a password.
"Active" as a count of visits	Not built and deliberately so. Beacon does not record when somebody opens the app, and the screen says what the number does mean instead of implying otherwise.

Bulk invitations, as built

Inviting twenty-five people one form at a time is not a workflow. The design was three stages so each could ship on its own; the first two are done and the third is not.

Stage	What it does	The rule it must not break
1. Paste a list, built	Paste any number of addresses, choose one role for the batch, see every row parsed in a preview, then send. Each row reports its own result.	Nothing is sent until the Director has seen the preview. Duplicates, malformed addresses and people who are already members are flagged before sending, not after.
2. Drop a file, built	Drag a spreadsheet onto the screen. The app finds the email, name and role columns and shows what it found.	Detection is always shown and always correctable. A mis-read role column would invite twenty-five people as Directors, and that is not a mistake you can take back.
3. Suggested pairing, not built	After a batch of Explorers, propose which Guide takes whom, and show the whole proposal for approval.	The cap of five is respected, and nobody is ever paired silently. A pairing is a relationship between two people, not a row in a table.

> **NOTE** · The one sentence to keep
>
> The app's limit is not servers, it is Guides. Everything about running this well follows from that: recruit a Guide, train them, pair them with up to five people, and watch the number of Explorers waiting for one.